

API Testing & Quality Assurance

Nhóm 10

Kim tự tháp **Kiểm thử**

Phân loại Kiểm thử



Unit Test

Kiểm tra các hàm độc lập. Ví dụ: Logic bấm mật khẩu, hoặc các rule Validation của MongoEngine Model.



Integration Test

Đánh giá sự kết nối. Đảm bảo Controller giao tiếp đúng với Database (MongoDB) và trả về JSON chuẩn.



Performance Test

Đo lường sức chịu tải (Load/Stress Testing).



Contract Test

Kiểm chứng API phản hồi chính xác theo bản hợp đồng OpenAPI Spec đã được thống nhất với Frontend.

Unit Testing với Pytest

- Sử dụng thư viện **pytest** kết hợp **mongomock** để giả lập Database, tránh làm bẩn dữ liệu thật.
- Tốc độ thực thi cực nhanh (vài mili-giây), giúp developer phát hiện lỗi logic ngay trong quá trình code.
- Chỉ cần viết test một lần, đảm bảo các hàm xử lý dữ liệu của MongoEngine luôn hoạt động đúng khi refactor.

```
@pytest.fixture
def client():
    disconnect()

    connect(
        db='mongoenginetest',
        mongo_client_class=mongomock.MongoClient
    )

    app.config['TESTING'] = True
```

```
def test_get_empty_books(client, auth_headers):
    response = client.get('/api/v1/books', headers=auth_headers)

    assert response.status_code == 200

    data = response.json
    assert isinstance(data.get("data"), list)
    assert len(data.get("data")) == 0
```

```
tests/test_books.py::test_get_empty_books PASSED
tests/test_books.py::test_create_book_success PASSED
tests/test_books.py::test_create_book_missing_field PASSED
```

Tự động hóa với Postman

Hệ sinh thái Postman & Newman



Test Scripts (Assertions)

Sử dụng JavaScript để tự động kiểm tra Status Code (200, 201) và xác thực cấu trúc dữ liệu JSON trả về.



Environment Variables

Tự động trích xuất access token từ API Login và lưu vào biến môi trường để dùng cho các request sau.



Collection Runner

Thực thi hàng loạt API theo một kịch bản định sẵn: Login
-> Lấy sách -> Thêm sách -> Cập nhật -> Xóa.



Newman CLI

Chạy bộ test Postman trực tiếp từ Command Line.

Xây dựng Kịch bản Test

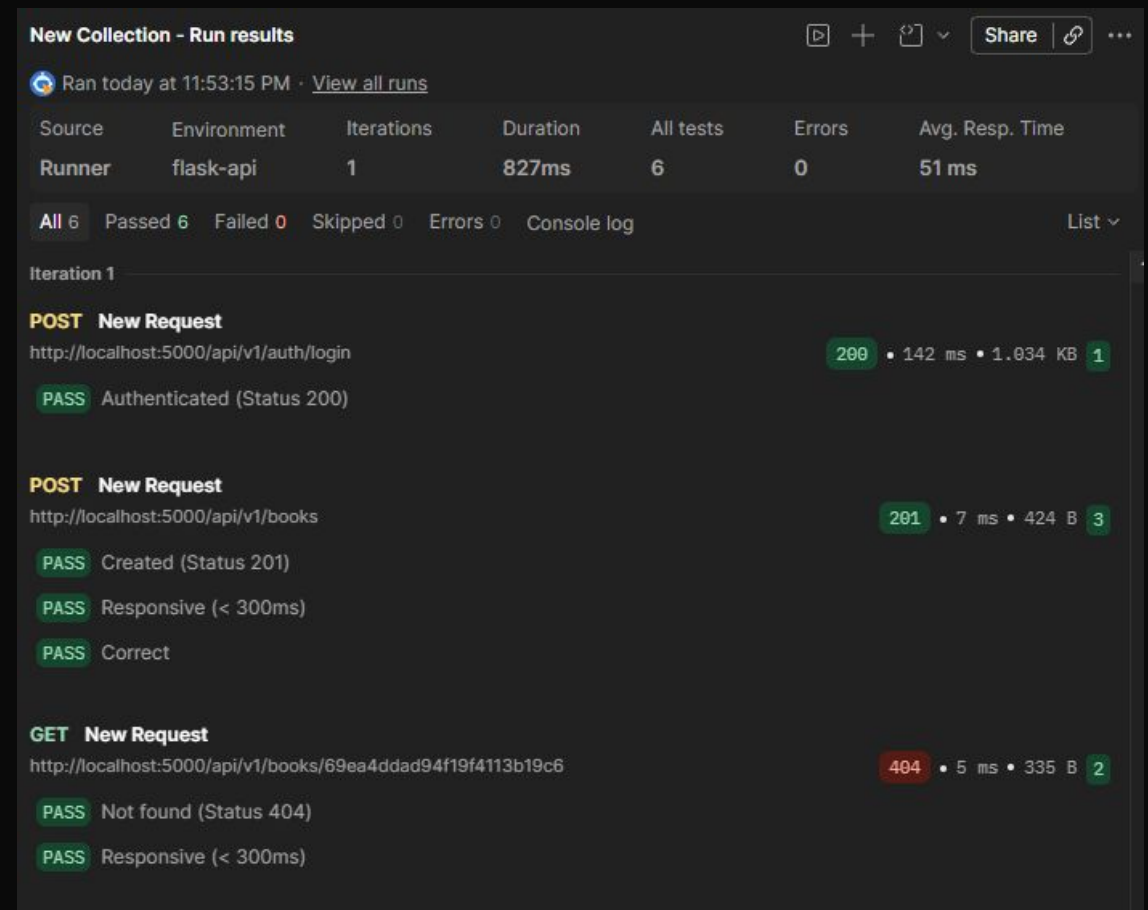
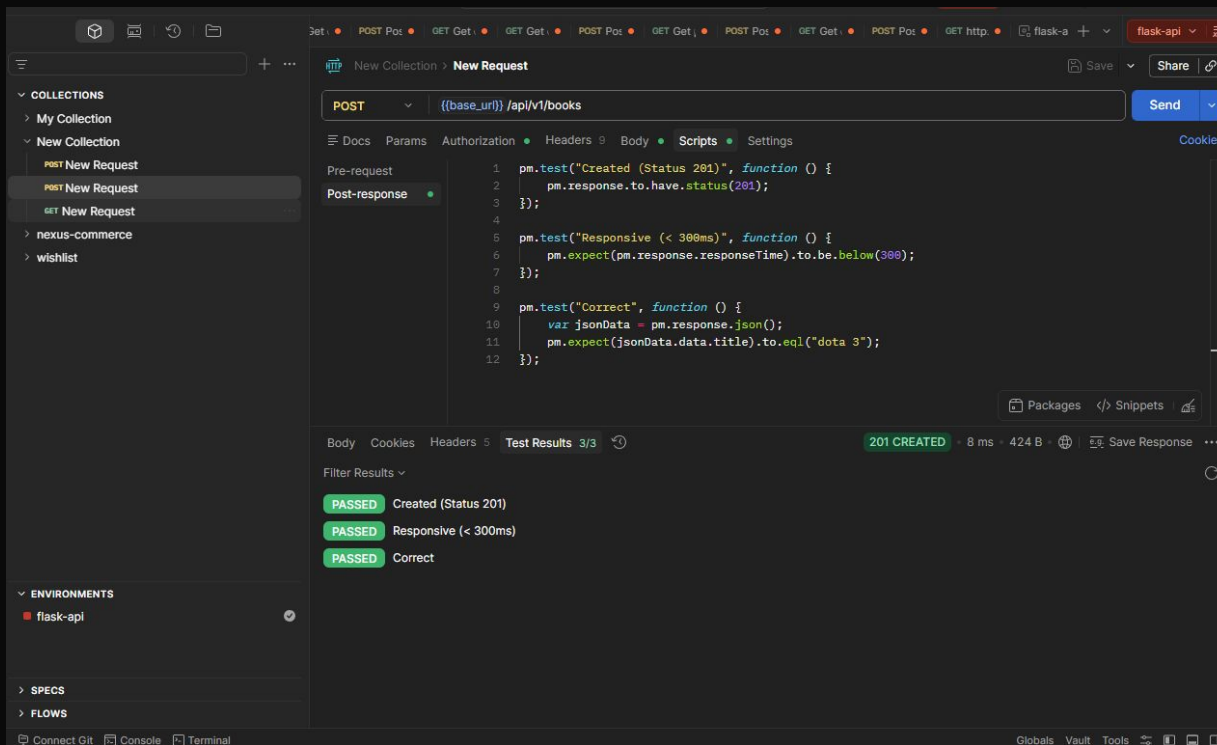
1. Happy Path (Kịch bản thành công)

- **POST /login:** Gửi đúng thông tin, assert status 200, lưu token.
- **POST /books:** Gửi token kèm Body hợp lệ, assert status 201 Created.
- **GET /books/{id}:** Gọi lại ID vừa tạo, assert dữ liệu trả về khớp hoàn toàn.

2. Negative Path (Kịch bản lỗi)

- **Gửi sai Token:** Test API xóa sách với Token giả, assert status 401 Unauthorized.
- **Thiếu trường bắt buộc:** Gửi payload tạo sách không có `title`, assert status 400 Bad Request.
- **Sai định dạng ID:** Tìm kiếm với Mongo Object ID sai, assert status 404 Not Found.

Xây dựng Kịch bản Test bằng Postman



Performance Testing

Các chỉ số Đo lường cốt lõi

Đánh giá qua Load Testing Tools (k6)

- **Response Time (Độ trễ):** Thời gian từ lúc Client gửi request đến lúc nhận phản hồi. Mục tiêu P95 < 200ms.
- **Error Rate (Tỷ lệ lỗi):** Phần trăm request bị thất bại (mã 5xx). Đòi hỏi < 1% dưới mức tải cao.
- **Throughput (Thông lượng):** Số lượng Requests Per Second (RPS) tối đa hệ thống xử lý được trước khi nghẽn cổ chai.

```
# k6 run perf.js

TOTAL RESULTS

checks_total.....: 1535    298.202235/s
checks_succeeded...: 100.00% 1535 out of 1535
checks_failed.....: 0.00%   0 out of 1535

✓ status is 200

HTTP
http_req_duration.....: avg=63.57ms min=5.51ms med=60.46ms max=157.14ms p(90)=74.13ms p(95)=86.96ms
{ expected_response:true }...: avg=63.57ms min=5.51ms med=60.46ms max=157.14ms p(90)=74.13ms p(95)=86.96ms
http_req_failed.....: 0.00%   0 out of 1535
http_reqs.....: 1535    298.202235/s

EXECUTION
iteration_duration.....: avg=165.24ms min=123.81ms med=161.6ms max=277.14ms p(90)=175.74ms p(95)=191.99ms
iterations.....: 1535    298.202235/s
vus.....: 50    min=50    max=50
vus_max.....: 50    min=50    max=50

NETWORK
data_received.....: 2.9 MB 566 kB/s
data_sent.....: 702 kB 136 kB/s
```

Viết load test script

```
import http from 'k6/http';  
import { check, sleep } from 'k6';  
  
export const options = {  
  vus: 50,           // concurrent users  
  duration: '5s',  
  thresholds: {  
    http_req_duration: ['p(95)<200'], // ms  
    http_req_failed: ['rate<0.01'],  
  },  
};  
  
export default function () {  
  const url = 'http://localhost:5000/api/v1/books';  
  const jwt = 'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJmcmVzaCI6ZmFsc2UsIm51dWUiOiJkaWE',  
  
  const params = { headers: { 'Authorization': 'Bearer ' + jwt } };  
  const res = http.get(url, params);  
  
  check(res, {  
    'status is 200': (r) => r.status === 200,  
  });  
  
  // sleep 0.5s after each response  
  sleep(0.5);  
}
```

- **Response Time (Độ trễ):** http_req_duration
- **Error Rate (Tỷ lệ lỗi):** http_req_failed
- **Maximum Throughput (Thông lượng tối đa):** 100 req/s
(vus * sleepTime)

Tích hợp QA vào vòng đời phát triển

